

3. Create a cursor for the connection with the following command:
4. Execute any SQL query using this cursor as shown below—here the outputs in terms of 1L or 2L show a number of rows affected by this query:

```
>>> cursor = conn.cursor()
>>> cursor.execute('Create database Library')
1L    // 1L Indicates how many rows affected
>>> cursor.execute('use Library')
>>> table='create table books(book_accno char(30) primary key,book_name
char(50),no_of_copies int(5),price int(5))'
>>> cursor.execute(table)
0L
```

5. Finally, fetch the result set and iterate over this result set. In this step, the user can fetch the result sets as shown below:

```
>>> cursor.execute('select * from books')
2L
>>> cursor.fetchall()
[('Py9098', 'Programming With Python', 100L, 50L), ('Py9099', 'Programming
With Python', 100L, 50L)]
```

In this example, the `fetchall()` function is used to fetch the result sets.

More SQL operations

We can perform all SQL operations with Python DB-API. Insert, delete, aggregate and update queries can be illustrated as follows.

1. Insert SQL Query

```
>>> cursor.execute('insert books values (%s,%s,%s,%s)', ('Py9098', 'Programmin
g With Python', 100, 50))
1L    // Rows affected.
>>> cursor.execute('insert books values (%s,%s,%s,%s)', ('Py9099', 'Programmin
g With Python', 100, 50))
1L    // Rows affected.
```

If the user wants to insert duplicate entries for a book's accession number, the Python DB-API will show an error as it is the primary key. The following example illustrates this:

```
>>> cursor.execute('insert books values (%s,%s,%s,%s)', ('Py9099', 'Programmin
g With Python', 100, 50))
>>> cursor.execute('insert books values (%s,%s,%s,%s)', ('Py9098', 'Programmin
g With Python', 120, 100))
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
  File "/usr/lib/python2.3/site-packages/MySQLdb/cursors.py", line 95, in
execute
    return self._execute(query, args)
  File "/usr/lib/python2.3/site-packages/MySQLdb/cursors.py", line 114, in
_execute
    self.errorhandler(self, exc, value)
  File "/usr/lib/python2.3/site-packages/MySQLdb/connections.py", line 33, in
defaulterrorhandler
    raise errorclass, errorvalue
_mysql_exceptions.IntegrityError: (1062, 'Duplicate entry 'Py9098' for key 1')
```

2. The **Update SQL** query can be used to update existing

records in the database as shown below:

```
>>> cursor.execute('update books set price=%s where no_of_
copies<=%s', [60, 101])
2L
>>> cursor.execute('select * from books')
2L
>>> cursor.fetchall()
[('Py9098', 'Programming With Python', 100L, 60L), ('Py9099', 'Programming
With Python', 100L, 60L)]
```

3. The **Delete SQL** query can be used to delete existing records in the database as shown below:

```
>>> cursor.execute('delete from books where no_of_copies<=%s', [101])
2L
>>> cursor.execute('select * from books')
0L
>>> cursor.fetchall()
[]
>>> cursor.execute('select * from books')
3L
>>> cursor.fetchall() [('Py9099', 'Python-Cookbook', 200L, 90L), ('Py9098',
'Programming With Python', 100L, 50L), ('Py9097', 'Python-Nut shell', 300L,
80L)]
```

4. Aggregate functions can be used with Python DB-API in the database as shown below:

```
>>> cursor.execute('select * from books')
4L
>>> cursor.fetchall()
[('Py9099', 'Python-Cookbook', 200L, 90L), ('Py9098', 'Programming With
Python', 100L, 50L), ('Py9097', 'Python-Nut shell', 300L, 80L), ('Py9096', 'Python-
Nut shell', 400L, 90L)]
>>> cursor.execute('select sum(price), avg(price) from books where
book_name='Python-Nut shell')
1L
>>> cursor.fetchall()
[(170.0, 85.0.)]
```

I hope you'll now enjoy the power of Python for database programming! 

References

- The Python home page: www.python.org
- Programming Python by Mark Lutz, O'Reilly Publications
- Accessing Databases using Python DBAPI-2.0 by Federico Di Gregorio: itnd.org/pub/software/psycapp/dbapi20programming.pdf

By: **Suhas A. Desai**

The author works at Tech Mahindra. He writes on open source and security. In his free time, he volunteers for social causes.